AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

PROGRAMOWANIE URZĄDZEŃ MOBILNYCH

RoadCracks

Autor:
Jakub Pyrdoł
Tymoteusz Zaczyk

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **4**

1.1. Cel projektu i rozwiązywany problem 4

1.2. Odbiorcy i inspiracja sposobem prezentacji danych 4

1.3. Podstawowy zakres funkcjonalny 4

1.4. Źródła danych wykorzystywane w aplikacji 5

1.5. Założenia dotyczące pomiaru i obsługi 6

2. Określenie wymagań szczegółowych **7**

2.1. Dane wejściowe i obsługa czujników 7

2.2. Wykrywanie zdarzeń i ocena odcinków drogi 7

2.3. Interfejs i obsługa aplikacji 8

2.4. Sytuacje niepożądane i wymagania jakościowe 9

2.5. Możliwości dalszego rozwoju 9

3. Projektowanie **11**

3.1. Przyjęte technologie i organizacja projektu 11

3.2. Struktura kodu i organizacja stanu aplikacji 11

3.3. Nawigacja i podział na zakładki 12

3.4. Mapa i obsługa lokalizacji 13

3.5. Informacje pogodowe na mapie 13

3.6. Ekran główny, alerty, profil i zapis lokalny 14

3.7. Motyw graficzny i dalsze projektowanie 15

4. Implementacja **16**

5. Testowanie **17**

6. Podręcznik użytkownika **18**

Literatura **19**

Spis rysunków **20**

Spis tabel **21**

Spis listingów **22**

1. Ogólne określenie wymagań

1.1. Cel projektu i rozwiązywany problem

Celem projektu jest stworzenie aplikacji mobilnej RoadCracks dla systemu Android, napisanej w języku Kotlin. Aplikacja będzie służyła do wykrywania i mapowania nierówności nawierzchni na podstawie danych rejestrowanych przez smartfon podczas jazdy. Głównym wynikiem jej działania będzie mapa z odcinkami dróg oznaczonymi kolorami odpowiadającymi oszacowanemu stopniowi ich uszkodzenia.

Projekt odpowiada na potrzebę uzyskania informacji o jakości nawierzchni na pokonywanej trasie. Pojedyncza informacja o wystąpieniu dziury nie opisuje całego odcinka drogi, dlatego RoadCracks ma uwzględniać zarówno nasilenie wstrząsów, jak i częstość ich występowania. Pozwoli to odróżnić odcinek z pojedynczą nierównością od drogi, na której niekorzystne zdarzenia powtarzają się na większej długości.

Ocena będzie miała charakter orientacyjny i będzie wynikała z pomiarów telefonu. Aplikacja nie będzie określała rzeczywistej głębokości ubytku, lecz poziom nasilenia zarejestrowanego zdarzenia. Dokładność wyniku będzie zależeć między innymi od prędkości przejazdu, właściwości pojazdu, mocowania urządzenia i jakości lokalizacji.

1.2. Odbiorcy i inspiracja sposobem prezentacji danych

RoadCracks będzie przeznaczona przede wszystkim dla kierowców, którzy chcą sprawdzić stan nawierzchni na pokonywanej trasie. Przejrzysta mapa powinna ułatwiać rozpoznanie miejsc, w których pomiary wskazują na większe nierówności, oraz odcinków, dla których nie zebrano jeszcze wystarczających danych.

Inspiracją dla sposobu prezentacji informacji jest aplikacja Yanosik [1], wykorzystująca mapę i informacje o zdarzeniach drogowych. W RoadCracks głównym przedmiotem prezentacji będzie jakość nawierzchni oraz położenie wykrytych nierówności i potencjalnych przeszkód.

1.3. Podstawowy zakres funkcjonalny

Docelowe działanie aplikacji będzie obejmowało cały przebieg pomiaru: od jego świadomego uruchomienia przez użytkownika, przez zbieranie danych, aż po zapis i prezentację wyniku. W podstawowym zakresie przewiduje się:

- rozpoczęcie i zakończenie rejestracji przejazdu;

- zapis przebiegu trasy i powiązanie wykrytych zdarzeń z lokalizacją;
- wykrywanie wstrząsów oraz oszacowanie ich nasilenia od lekkiego do krytycznego;
- oznaczanie potencjalnych przeszkód rozpoznanych na podstawie gwałtownego manewru omijania;
- ocenę odcinków drogi i przedstawienie wyniku za pomocą kolorów na mapie;
- dostosowanie jasnego lub ciemnego motywu do warunków oświetlenia.

Zdarzenia wynikające wyłącznie z gwałtownego manewru powinny być oznaczane oddzielnie od zdarzeń wykrytych na podstawie wstrząsu. Samo ominięcie przeszkody nie będzie traktowane jako potwierdzenie uszkodzenia nawierzchni. Odcinki bez wystarczających pomiarów powinny mieć odrębne oznaczenie, aby brak danych nie był interpretowany jako dobry stan drogi.

1.4. Źródła danych wykorzystywane w aplikacji

Projekt zakłada wykorzystanie czterech źródeł danych dostępnych w smartfonie. Są to moduł lokalizacji GPS oraz trzy czujniki telefonu:

- **GPS** umożliwi określenie położenia i prędkości, zapis trasy oraz przypisanie zdarzeniom współrzędnych geograficznych.
- **Akcelerometr** posłuży do wykrywania krótkotrwałych wstrząsów związanych z przejazdem przez nierówności. Analizie będzie podlegać przede wszystkim składowa przyspieszenia odpowiadająca kierunkowi pionowemu.
- **Żyroskop** dostarczy informacji o ruchu obrotowym telefonu. W połączeniu z prędkością z GPS umożliwi poszukiwanie wzorca gwałtownego manewru omijania przeszkody.
- **Czujnik światła** posłuży do automatycznego przełączania motywu interfejsu pomiędzy jasnym i ciemnym.

Dane ruchowe i lokalizacyjne powinny być łączone według czasu rejestracji, tak aby wykryte zdarzenie zostało przypisane do miejsca jego wystąpienia. Czujnik światła będzie wpływał na sposób prezentacji interfejsu, a nie na ocenę nawierzchni.

1.5. Założenia dotyczące pomiaru i obsługi

Podczas rejestracji telefon powinien być stabilnie zamocowany w uchwycie. Przed rozpoczęciem pomiaru aplikacja powinna ustalić jego położenie, aby analizować przyspieszenie w kierunku pionowym niezależnie od orientacji urządzenia. Poruszanie telefonem w ręce nie powinno być traktowane jako wiarygodny pomiar stanu nawierzchni.

Interfejs powinien być prosty i czytelny, z wyraźnym dostępem do mapy oraz funkcji rozpoczęcia i zakończenia pomiaru. Po uruchomieniu rejestracji zbieranie danych powinno odbywać się automatycznie. Szczegółowe wymagania funkcjonalne przedstawiono w rozdziale 2, a opis przygotowanego dotychczas prototypu i kierunek dalszego projektowania w rozdziale 3.

2. Określenie wymagań szczegółowych

2.1. Dane wejściowe i obsługa czujników

GPS. Aplikacja powinna odczytywać współrzędne geograficzne, prędkość, czas pomiaru oraz dostępną informację o dokładności lokalizacji. Dane te posłużą do rysowania trasy i umieszczania wykrytych zdarzeń na mapie. Wstrząs należy powiązać z odczytem lokalizacji odpowiadającym czasowi jego wystąpienia. Przy słabym sygnale aplikacja powinna sygnalizować obniżoną dokładność, zamiast przedstawiać położenie zdarzenia jako pewne.

Akcelerometr. Aplikacja powinna odczytywać przyspieszenie w trzech osiach i wyznaczać jego składową pionową. Analiza osi Z jako kierunku pionowego jest dopuszczalna tylko przy odpowiednim ustawieniu telefonu. Po uwzględnieniu orientacji urządzenia i składowej grawitacyjnej algorytm powinien wykrywać krótkie, wyraźne wstrząsy oraz nadawać im poziom nasilenia od lekkiego do krytycznego. Wstrząs może pochodzić od dziury, progu zwalniającego lub studzienki, dlatego sam pomiar nie powinien przesądzać o rodzaju obiektu.

Żyroskop. Aplikacja powinna analizować prędkość kątową telefonu w celu wykrywania sekwencji gwałtownego skrętu w jedną stronę i powrotu w przeciwną. Jeżeli taki wzorzec pojawi się podczas jazdy, przy odpowiedniej prędkości wskazanej przez GPS i bez istotnego wstrząsu, miejsce powinno zostać oznaczone jako potencjalna ominięta przeszkoda. Jest to przybliżone rozpoznanie manewru omijania, określanego jako manewr łosia. Żyroskop mierzy ruch telefonu, dlatego stabilne mocowanie urządzenia jest warunkiem wiarygodnej analizy.

Czujnik światła. Aplikacja powinna odczytywać natężenie oświetlenia i automatycznie włączać motyw ciemny przy słabym świetle, a jasny przy silnym oświetleniu. Zmiana powinna następować dopiero po utrzymaniu się warunków przez krótki czas, aby zapobiec częstemu przełączaniu motywu, na przykład podczas przejazdu pod drzewami. Użytkownik powinien mieć także możliwość ręcznego wyboru motywu.

2.2. Wykrywanie zdarzeń i ocena odcinków drogi

Pomiary z różnych źródeł powinny być łączone według czasu ich rejestracji. Przed rozpoczęciem przejazdu aplikacja powinna wykonać krótką kalibrację położenia telefonu. Algorytm powinien ograniczać wpływ drobnych drgań, ruchów telefonu i wielokrotnego wykrywania tego samego wstrząsu. Wykrywanie zdarzeń drogowych po-

winno działać podczas jazdy; odczyty z postoju nie powinny automatycznie tworzyć zgłoszeń.

Dla każdego zdarzenia należy zapisać jego czas, położenie, prędkość przejazdu, typ wykrycia i oszacowany poziom nasilenia. Zdarzenia należy przypisywać do odcinków trasy, uwzględniając dokładność GPS. Kolor odcinka powinien zależeć od liczby zdarzeń w odniesieniu do jego długości oraz ich siły. Potencjalny manewr ominięcia przeszkody nie powinien samodzielnie oznaczać krytycznego uszkodzenia nawierzchni.

Przyjętą skalę prezentacji stanu nawierzchni przedstawiono w tabeli 2.1. Poszczególne kolory odpowiadają oszacowanemu nasileniu nierówności, a kolor szary oznacza brak wystarczających danych. Progi wykrywania, granice poziomów nasilenia i sposób łączenia pomiarów zostaną dobrane podczas testów, ponieważ wynik zależy między innymi od prędkości, mocowania telefonu i właściwości pojazdu.

Tab. 2.1. Skala oceny odcinków drogi

Kolor	Znaczenie
Zielony	Brak istotnych nierówności w zebranych pomiarach.
Żółty	Niewielkie nierówności i lekkie wstrząsy.
Pomarańczowy	Nierówności o średnim nasileniu.
Czerwony	Poważne nierówności i silne wstrząsy.
Bordowy	Najwyższy poziom nasilenia, oceniony jako krytyczny.
Szary	Brak danych wystarczających do oceny odcinka.

Brak zgłoszeń na niezmierzonym odcinku nie może być utożsamiany z dobrym stanem nawierzchni. Potencjalne ominięte przeszkody powinny być prezentowane za pomocą osobnego znacznika, odróżniającego je od zdarzeń wykrytych na podstawie wstrząsu.

2.3. Interfejs i obsługa aplikacji

Główny ekran powinien zawierać mapę, bieżącą lokalizację użytkownika, pokonywaną trasę oraz kolorowe oznaczenia ocenionych odcinków. Widoczna legenda powinna wyjaśniać znaczenie kolorów i znaczników. Ekran powinien również pokazywać prędkość, stan pomiaru i dostępność lokalizacji.

- **Rozpocznij pomiar** sprawdza uprawnienia i czujniki, uruchamia kalibrację, a następnie rejestrację przejazdu.
- **Zakończ pomiar** zatrzymuje rejestrację i zapisuje przejazd oraz wykryte zdarzenia.
- **Moja lokalizacja** ustawia mapę na bieżącej pozycji użytkownika.
- **Ustawienia** otwiera wybór motywu i informacje o dostępności czujników.

Dotknięcie oznaczenia zdarzenia powinno wyświetlać jego typ, czas rejestracji i poziom nasilenia, a wybór odcinka drogi powinien pokazywać jego ocenę. Podczas aktywnego pomiaru aplikacja powinna automatycznie aktualizować trasę i oznaczenia oraz zmieniać motyw zgodnie z ustawieniami. Komunikaty o problemach powinny być krótkie i jednoznaczne.

2.4. Sytuacje niepożądane i wymagania jakościowe

Przed rozpoczęciem pomiaru aplikacja powinna poprosić o dostęp do lokalizacji i wyjaśnić jego cel. Odmowa uprawnienia lub wyłączenie lokalizacji powinny skutkować komunikatem o braku możliwości przypisania zdarzeń do mapy. Przy chwilowej utracie GPS aplikacja nie powinna przypisywać nowych zdarzeń do ostatniej znanej pozycji jako aktualnej.

Brak akcelerometru powinien wyłączać wykrywanie wstrząsów, a brak żyroskopu wykrywanie manewrów omijania. Brak czujnika światła powinien pozostawiać możliwość ręcznego wyboru motywu. Ograniczona funkcjonalność musi być widoczna dla użytkownika. W razie utraty połączenia z internetem pomiary powinny być przechowywane lokalnie; jeżeli nie uda się pobrać mapy, aplikacja powinna poinformować o ograniczeniu jej widoku.

Interfejs powinien pozostawać płynny podczas zbierania danych, a pomiary powinny być zapisywane na bieżąco, aby ograniczyć ich utratę po przerwaniu pracy aplikacji. Odczyt czujników należy zakończyć po zakończeniu pomiaru, ograniczając zużycie baterii. Dostęp do danych o lokalizacji powinien być ograniczony do funkcji aplikacji, a rejestrowanie trasy powinno odbywać się po świadomym uruchomieniu pomiaru przez użytkownika.

2.5. Możliwości dalszego rozwoju

W kolejnych wersjach możliwe będzie współdzielenie zgłoszeń między użytkownikami, potwierdzanie nierówności na podstawie wielu przejazdów, uwzględnianie wieku

pomiarów oraz proponowanie tras o lepszej nawierzchni. Funkcje te stanowią kierunki rozwoju i nie są wymagane w podstawowym zakresie projektu.

3. Projektowanie

3.1. Przyjęte technologie i organizacja projektu

Aplikację RoadCracks przygotowano w języku Kotlin jako projekt Android z modułem `app`. Konfigurację budowania zapisano w plikach Gradle Kotlin DSL z rozszerzeniem `.kts`, a wersje bibliotek zebrano w pliku `libs.versions.toml` w katalogu `gradle`. W przekazanej wersji projektu minimalny poziom API wynosi 24, natomiast `compileSdk` i `targetSdk` mają wartość 37. Są to ustawienia projektu; zgodność działania na urządzeniach wymaga osobnego sprawdzenia.

Interfejs wykorzystuje Jetpack Compose [2], komponenty Material 3 oraz bibliotekę rozszerzonych ikon. Ekrany i elementy wielokrotnego użytku zapisano jako funkcje oznaczone adnotacją `@Composable`. Do wyświetlania mapy wybrano MapLibre Native, zintegrowane z Compose przez `AndroidView`. Mapa korzysta ze stylu Liberty udostępnianego przez OpenFreeMap, a dane pogodowe pobierane są z Open-Meteo. Adresy tych usług są już zapisane w kodzie aplikacji.

Obecna wersja obejmuje nawigację między pięcioma zakładkami, mapę z pozycją użytkownika, kartę pogody, edycję profilu oraz lokalną obsługę przykładowych alertów. Funkcje pomiaru nierówności opisane w rozdziałach 1 i 2 pozostają docelowym zakresem rozwoju.

3.2. Struktura kodu i organizacja stanu aplikacji

Punktem wejścia jest klasa `MainActivity`, dziedzicząca po `ComponentActivity`. W metodzie `onCreate` wywołanie `setContent` uruchamia funkcję `RoadCracksApp` wewnątrz `RoadCracksTheme`. Główny układ i obsługę stanu wydzielono do pliku `RoadCracksApp.kt`; funkcja `RoadCracksMainLayout` jest dodatkowym wywołaniem tego samego układu.

Kod podzielono zgodnie z odpowiedzialnością poszczególnych elementów:

- `ui/screens` zawiera osobne pliki pięciu ekranów: głównego, mapy, mapowania, alertów i profilu;
- `ui/components` zawiera między innymi widok mapy, kartę pogody, kartę alertu, awatar i karty statystyk;
- `data` obejmuje modele profilu, alertu i pogody, klasę zapisu lokalnego oraz repozytorium danych pogodowych;

- `utils` zawiera funkcje obsługi lokalizacji, wczytywania zdjęć i formatowania dat;
- `ui/theme` zawiera kolory, typografię oraz wspólny motyw aplikacji.

W funkcji `RoadCracksApp` przechowywane są między innymi profil, obserwowane miejsce, lista alertów i stan pływającego komunikatu. Ekrany otrzymują potrzebne dane i funkcje obsługujące działania użytkownika, na przykład zapis profilu lub usunięcie alertu. Aktualizacja wspólnego stanu pozwala odświeżać podsumowanie na ekranie głównym po zmianie danych w innej zakładce. Wybraną zakładkę i tekst wyszukiwania zachowuje mechanizm `rememberSaveable`.

3.3. Nawigacja i podział na zakładki

Układ aplikacji oparto na komponencie `Scaffold`. W jego dolnej części znajduje się `NavigationBar` z pozycjami `NavigationBarItem` [3]. Przyjęto następujący podział:

- **Główna** wyświetla podsumowanie, statystyki, obserwowane miejsce oraz ostatnie alerty;
- **Mapa** udostępnia mapę, pozycję użytkownika, informacje o lokalizacji i pogodzie;
- **Mapuj** zawiera wstępny widok trybu rejestrowania trasy;
- **Alert** umożliwia zapis obserwowanego miejsca i zarządzanie lokalnymi alertami;
- **Profil** umożliwia zmianę danych użytkownika i zdjęcia profilowego.

Wybrana zakładka jest określona przez zmienną `selectedItem`. Domyślna wartość parametru `initialScreen` wynosi 0, dlatego aplikacja uruchamia ekran Główna. Dotknięcie pozycji dolnego paska zmienia indeks, a instrukcja `when` wyświetla odpowiedni ekran. Przyciski ekranu głównego umożliwiają również bezpośrednie przejście do mapowania lub alertów. Nawigacja jest realizowana przez zmianę stanu, bez osobnego grafu nawigacji.

Wyłącznie w zakładce Mapa nad paskiem nawigacji pojawia się pole wyszukiwania adresu. Można wprowadzić tekst i wyczyścić go przyciskiem. W obecnym kodzie wartość pola nie jest przekazywana do mapy ani do usługi wyszukiwania adresów.

3.4. Mapa i obsługa lokalizacji

Ekran `MapScreen` korzysta z komponentu `UserLocationMap`, który łączy mapę, kartę pogody i panel komunikatów o lokalizacji. Komponent `MapLibreMapView` osadza natywny `MapView` i przekazuje mu aktualne położenie. Po wczytaniu stylu pozycja jest przedstawiana jako własny znacznik oparty na źródle GeoJSON. Początkowy widok obejmuje obszar Polski, a po uzyskaniu pozycji mapa jest przybliżana do użytkownika. Kolejne odczyty aktualizują znacznik; ponowne wyśrodkowanie można wywołać przyciskiem *Moja pozycja*.

Manifest zawiera uprawnienia dostępu do internetu, stanu sieci oraz lokalizacji dokładnej i przybliżonej. Komponent mapy sprawdza zgodę użytkownika i stan lokalizacji w telefonie. Przycisk panelu, zależnie od sytuacji, uruchamia prośbę o uprawnienia, otwiera ustawienia aplikacji lub lokalizacji, ponawia wczytanie mapy albo ustawia widok na bieżącej pozycji. Oddzielne komunikaty opisują oczekiwanie na pozycję, lokalizację przybliżoną i błędy pobierania danych.

Funkcja `userLocationUpdates` udostępnia odczyty jako strumień `Flow`, korzystając z mechanizmu lokalizacji MapLibre. Żądany odstęp aktualizacji wynosi 5 s, z minimalnym odstępem 2 s i priorytetem wysokiej dokładności. Funkcja pomocnicza sprawdza zakres współrzędnych i czas pomiaru; odczyty starsze niż 120 s są odrzucane. Odbieranie danych jest powiązane z aktywnym stanem ekranu, a zamknięcie strumienia usuwa nasłuchiwanie. Cykl życia natywnego widoku mapy jest obsługiwany oddzielnie, łącznie ze zwalnianiem jego zasobów.

Mapa nie przedstawia jeszcze zapisanej trasy ani odcinków ocenionych według stanu nawierzchni. Boczne panele z napisami *200 m* i *Km/h* pozostają elementami demonstracyjnymi: ich dotknięcie wyświetla komunikat `Toast`. Nie pokazują one obliczonej odległości ani prędkości z lokalizacji.

3.5. Informacje pogodowe na mapie

W górnej części mapy umieszczono `MapWeatherCard`. Po uzyskaniu lokalizacji karta wyświetla nazwę okolicy, temperaturę, ikonę i opis warunków pogodowych. Repozytorium `WeatherRepository` pobiera temperaturę, kod pogody oraz informację o porze dnia z Open-Meteo przez połączenie HTTPS. Nazwa miejscowości jest ustalana za pomocą systemowego `Geocoder`; gdy nie uda się jej uzyskać, stosowany jest opis *Twoja okolica*.

Operacje sieciowe wykonuje się poza głównym wątkiem interfejsu. Repozytorium przechowuje ostatni odczyt w pamięci i może użyć go ponownie przez 15 min,

jeśli użytkownik pozostaje w odległości mniejszej niż 1 km od miejsca pobrania danych. Zastosowano także ograniczenie częstości ponawiania żądań. Dotknięcie karty pozwala zażądać odświeżenia, a brak lokalizacji, pobieranie i błąd mają osobne komunikaty. Starszy odczyt jest oznaczany jako *Ostatnie dane*. Karta zawiera również odnośnik do źródła danych.

Pogoda uzupełnia widok mapy. W obecnej wersji nie wpływa na ocenę nawierzchni ani na wybór motywu całej aplikacji.

3.6. Ekran główny, alerty, profil i zapis lokalny

Ekran Główna przedstawia powitanie z nazwą użytkownika, statystyki, obserwowane miejsce i maksymalnie trzy ostatnie alerty. Liczba wszystkich i nieprzeczytanych powiadomień wynika z aktualnej listy. Liczba zapisanych tras oraz długość zmapowanych dróg mają natomiast stałe wartości zerowe, ponieważ rejestrowanie przejazdu nie zostało jeszcze podłączone. Zakładka Mapuj zawiera tytuł trybu i przycisk *Rozpocznij Nagrywanie Trasy*, którego obsługa jest pusta.

W zakładce Alert użytkownik może wpisać i zapisać nazwę obserwowanego miasta lub ulicy. Przycisk *Pokaż przykładowy alert* tworzy testowy komunikat z identyfikatorem, opisem, miejscem i czasem utworzenia. Alert można usunąć, otworzyć jego szczegóły lub oznaczyć wszystkie komunikaty jako przeczytane. Otwarcie szczegółów oznacza dany wpis jako przeczytany. Po dodaniu przykładu nad aktualnym ekranem pojawia się karta `FloatingAlertCard`; można ją zamknąć lub przejść z niej do alertów, a po 8 s jest automatycznie ukrywana.

Pływająca karta jest elementem interfejsu aplikacji. Nie zastosowano mechanizmu systemowych powiadomień ani pobierania rzeczywistych ostrzeżeń drogowych z serwera. Zapisane obserwowane miejsce służy obecnie jako tekst wyświetlany w podsumowaniu i przykładowych alertach.

Ekran Profil udostępnia nazwę użytkownika, adres e-mail, wybór płci i zdjęcie. Zapis wymaga niepustej nazwy i wybranej płci; adres e-mail jest opcjonalny, lecz wpisana wartość musi mieć poprawny format. Zdjęcie wybiera się systemowym selektorem dokumentów. Aplikacja zachowuje uprawnienie do odczytu wybranego URI, wczytuje obraz poza głównym wątkiem i ogranicza jego rozmiar podczas dekodowania. Jeśli zdjęcia nie można odczytać, wyświetlany jest domyślny awatar. Profil ma charakter lokalny; kod nie zawiera logowania ani konta na serwerze.

Modele `UserProfile` i `RoadAlert` opisują dane przekazywane między ekranami. Klasa `LocalData` zapisuje profil, URI zdjęcia i obserwowane miejsce w `SharedPreferences`. Lista alertów jest przechowywana jako JSON i ograniczona do 50 wpisów, wraz

z informacją o ich przeczytaniu. Dane te są odczytywane przy tworzeniu głównego układu, dzięki czemu pozostają dostępne po ponownym uruchomieniu aplikacji. W tym mechanizmie nie są jeszcze zapisywane trasy ani pomiary czujników.

3.7. Motyw graficzny i dalsze projektowanie

Wspólny wygląd zapewnia `RoadCracksTheme`, zawierający jasny i ciemny zestaw kolorów oraz typografię. Wybór wariantu jest powiązany z motywem systemowym. Na urządzeniach z Androidem 12 lub nowszym kod umożliwia użycie dynamicznych kolorów. Projekt zawiera podglądy Compose dla poszczególnych zakładek; w podglądzie mapa jest zastępowana widokiem informacyjnym, a karta pogody pokazuje przykładowe dane.

Dalsze projektowanie obejmuje połączenie zakładki Mapuj z rozpoczęciem i zakończeniem pomiaru, kalibracją telefonu oraz zapisem przejazdu. Do przygotowania pozostają odczyty akcelerometru, żyroskopu i czujnika światła, łączenie zdarzeń z lokalizacją, klasyfikacja nierówności i kolorowanie odcinków zgodnie ze skalą z tabeli 2.1. Konieczne będzie również podłączenie wyszukiwania adresów i przełączania motywu według oświetlenia. Rozdział przedstawia zatem aktualny projekt interfejsu, mapy i danych lokalnych oraz wskazuje elementy potrzebne do realizacji wymagań pomiarowych.

4. Implementacja

5. Testowanie

6. Podręcznik użytkownika

Literatura

[1] Yanosik. *Yanosik – system ostrzeżeń drogowych*. URL: https://yanosik.pl/ (dostęp 28.10.2026).

[2] Android Developers. *Jetpack Compose*. URL: https://developer.android.com/compose (dostęp 28.10.2026).

[3] Android Developers. *Navigation bar*. URL: https://developer.android.com/develop/ui/compose/components/navigation-bar (dostęp 28.10.2026).

Spis rysunków

Spis tabel

2.1. Skala oceny odcinków drogi 8

Spis listingów